

Aula 5:

Range Query

Disciplina: Maratona de Programação 1

Profs. Edmilson Marmo e Luiz Olmes

edmarmo@unifei.edu.br, olmes@unifei.edu.br



Problema

- ▶ Antes de iniciar a aula, vamos tentar resolver o seguinte problema.
- ▶ Produto do Intervalo:
 - ▶ <https://judge.beecrowd.com/pt/problems/view/1301>

Nas aulas anteriores...

▶ **O QUE JÁ ESTUDAMOS?**

- ▶ Introdução à Maratona
- ▶ Problemas ad hoc
- ▶ Standard Template Library

▶ **OBJETIVOS:**

- ▶ Consulta a intervalos.
- ▶ Prefix sum array.
- ▶ Segment tree:
 - ▶ Construção
 - ▶ Consulta
 - ▶ Atualização

Range Query

- ▶ A tradução mais próxima para o termo “range query” é “consulta em um intervalo”.
 - ▶ Este tipo de consulta manipula partes de um conjunto de dados.
 - ▶ Subconjuntos.
- ▶ Considere um vetor A com N posições: `int A[N];`
- ▶ Dados dois valores i e j ($0 \leq i, j \leq N$), deseja-se obter:
 - ▶ `max(A[i], A[i + 1], ..., A[j-1], A[j])`
 - ▶ `min(A[i], A[i + 1], ..., A[j-1], A[j])`
 - ▶ `sum(A[i], A[i + 1], ..., A[j-1], A[j])`

Range Query

- ▶ Dados dois valores i e j ($0 \leq i, j \leq N$), deseja-se obter:
 - ▶ $\max(A[i], A[i + 1], \dots, A[j-1], A[j])$
 - ▶ $\min(A[i], A[i + 1], \dots, A[j-1], A[j])$
 - ▶ $\text{sum}(A[i], A[i + 1], \dots, A[j-1], A[j])$
- ▶ Estamos interessados em obter o valor máximo **dentro de um intervalo**.
 - ▶ Ou o valor mínimo, ou a soma dos elementos do intervalo.

Range Query

- ▶ Dados dois valores i e j ($0 \leq i, j \leq N$), deseja-se obter:
 - ▶ $\max(A[i], A[i + 1], \dots, A[j-1], A[j])$
 - ▶ $\min(A[i], A[i + 1], \dots, A[j-1], A[j])$
 - ▶ $\text{sum}(A[i], A[i + 1], \dots, A[j-1], A[j])$
- ▶ Estamos interessados em obter o valor máximo **dentro de um intervalo**.
 - ▶ Ou o valor mínimo, ou a soma dos elementos do intervalo.

- ▶ Como fazer?



Range Query

- ▶ Dado o vetor abaixo, queremos obter a soma de um intervalo.
 - ▶ Por exemplo, `sum(3, 6)`.

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
1	3	4	8	6	1	4	2

Range Query

- ▶ Dado o vetor abaixo, queremos obter a soma de um intervalo.
 - ▶ Por exemplo, `sum(3, 6)`.

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
1	3	4	8	6	1	4	2

- ▶ Primeira tentativa:
 - ▶ Um laço com início na posição 3 e final em 6, para percorrer o vetor e acumular a soma em uma variável.

Range Query

- ▶ Dado o vetor abaixo, queremos obter a soma de um intervalo.
 - ▶ Por exemplo, $\text{sum}(3, 6)$.

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
1	3	4	8	6	1	4	2

- ▶ Primeira tentativa:
 - ▶ Um laço com início na posição 3 e final em 6, para percorrer o vetor e acumular a soma em uma variável.

- ▶ Funciona?



Range Query

- ▶ Dado o vetor abaixo, queremos obter a soma de um intervalo.
 - ▶ Por exemplo, $\text{sum}(3, 6)$.

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
1	3	4	8	6	1	4	2

- ▶ Primeira tentativa:
 - ▶ Um laço com início na posição 3 e final em 6, para percorrer o vetor e acumular a soma em uma variável.
- ▶ **Funciona?** Sim, em tempo $O(n)$.



Range Query

- ▶ Dado o vetor abaixo, queremos obter a soma de um intervalo.
 - ▶ Por exemplo, $\text{sum}(3, 6)$.

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
1	3	4	8	6	1	4	2

- ▶ Primeira tentativa:
 - ▶ Um laço com início na posição 3 e final em 6, para percorrer o vetor e acumular a soma em uma variável.

- ▶ Boa solução?



Range Query

- ▶ Dado o vetor abaixo, queremos obter a soma de um intervalo.
 - ▶ Por exemplo, $\text{sum}(3, 6)$.

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
1	3	4	8	6	1	4	2

- ▶ Primeira tentativa:
 - ▶ Um laço com início na posição 3 e final em 6, para percorrer o vetor e acumular a soma em uma variável.
- ▶ Boa solução?
- ▶ Não. Para m consultas, se m for muito grande, resulta em $O(mn)$, que é ruim.

Range Query

- ▶ Dado o vetor abaixo, queremos obter a soma de um intervalo.
 - ▶ Por exemplo, $\text{sum}(3, 6)$.

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
1	3	4	8	6	1	4	2

- ▶ Segunda tentativa:
 - ▶ Realizar todas as somas, de uma só vez, no início do programa. Acessar apenas as posições inicial e final. O vetor de somas ficaria:

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
1	4	8	16	22	23	27	29

Vetor de soma de prefixo

- ▶ O problema anterior pode ser facilmente resolvido através de um *prefix sum array*.
- ▶ Cada valor no vetor de prefixo corresponde à soma dos valores anteriores do vetor original, até a posição atual.
 - ▶ Isto é, o valor da posição k é $\text{sum}(0, k)$.
- ▶ Um *prefix sum array* pode ser construído em tempo $O(n)$. A partir daí, pode-se calcular qualquer valor $\text{sum}(a, b)$ em tempo $O(1)$, da seguinte maneira:
 - ▶ $\text{sum}(a, b) = \text{sum}(0, b) - \text{sum}(0, a - 1)$.
 - ▶ onde, por definição, $\text{sum}(0, -1) = 0$.

Vetor de soma de prefixo

- ▶ Dado o vetor abaixo, queremos obter a soma de um intervalo.
 - ▶ Por exemplo, $\text{sum}(3, 6)$.

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
1	3	4	8	6	1	4	2

- ▶ Usando o vetor de prefixo:

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
1	4	8	16	22	23	27	29

- ▶ $\text{sum}(3, 6) = 19$.

Vetor de soma de prefixo

- ▶ Um *prefix sum array* pode ser construído em tempo $O(n)$.
 - ▶ Apenas 1 vez, no início do programa.
- ▶ A partir daí, pode-se calcular qualquer valor $\text{sum}(a, b)$ em tempo $O(1)$.
 - ▶ Para cada consulta.
- ▶ E se o vetor precisar ser atualizado? Isto é, o valor de algum elemento for alterado?
 - ▶ É possível dar suporte à operação de **atualização** (update) de forma **eficiente**?



Vetor de soma de prefixo

- ▶ Um *prefix sum array* pode ser construído em tempo $O(n)$.
 - ▶ Apenas 1 vez, no início do programa.
- ▶ A partir daí, pode-se calcular qualquer valor $\text{sum}(a, b)$ em tempo $O(1)$.
 - ▶ Para cada consulta.
- ▶ E se o vetor precisar ser atualizado? Isto é, o valor de algum elemento for alterado?
 - ▶ É possível dar suporte à operação de **atualização** (update) de forma **eficiente**?
 - ▶ **Não**. É necessário re-computar parte do vetor de prefixo: para **m** updates, $O(mn)$.

Segment tree

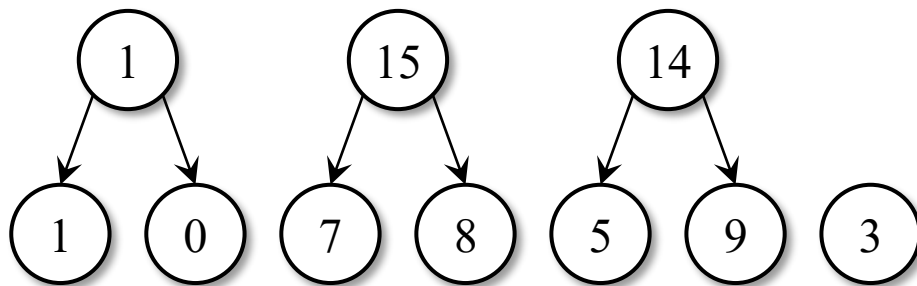
- ▶ Uma *segment tree* (árvore de segmentos) é uma estrutura que permite realizar duas operações em tempo $O(\log n)$:
 - ▶ Range query.
 - ▶ Updates.
- ▶ Trata-se de uma *árvore binária* onde os elementos em folha correspondem aos elementos do vetor.
 - ▶ Altura: $\log_2 (n + 1)$
- ▶ Os nós internos (índices) contém a informação de uma range query (soma, produto, mínimo, máximo...).

Segment tree



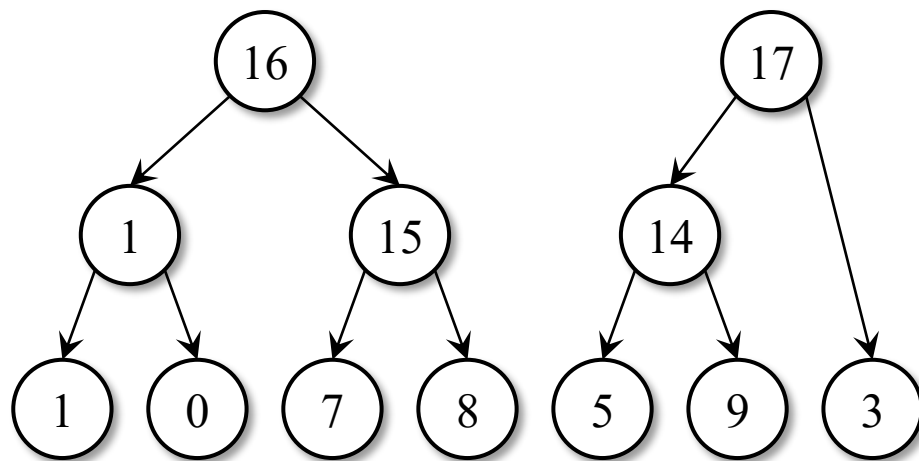
1	0	7	8	5	9	3
[0]	[1]	[2]	[3]	[4]	[5]	[6]

Segment tree



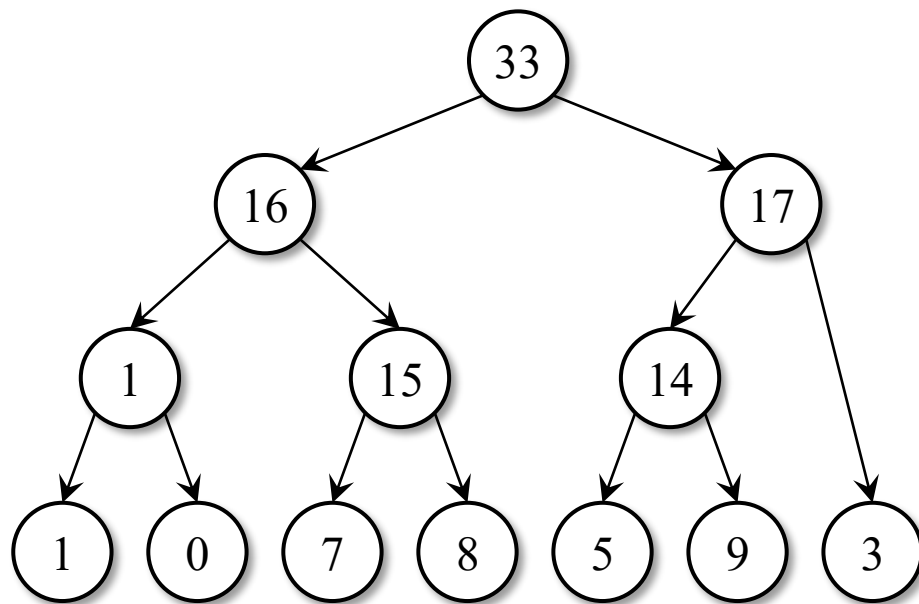
1	0	7	8	5	9	3
[0]	[1]	[2]	[3]	[4]	[5]	[6]

Segment tree



1	0	7	8	5	9	3
[0]	[1]	[2]	[3]	[4]	[5]	[6]

Segment tree



1	0	7	8	5	9	3
[0]	[1]	[2]	[3]	[4]	[5]	[6]

Segment tree

- ▶ Cada nó da árvore contém as seguintes informações:
 - ▶ **from**: início do segmento
 - ▶ **to**: final do segmento
 - ▶ **value**: valor da soma (ou outra range query).

```
1. struct segment_tree
2. {
3.     segment_tree *left, *right;
4.     int from, to, value;
5.
6.     segment_tree(int _from, int _to)
7.         : from(_from), to(_to), left(NULL), right(NULL), value(0) { }
8. };
```

Zero: elemento neutro da soma.

Segment tree

```
1.  segment_tree* build(const vector<int> &arr, int l, int r)
2.  {
3.      if (l > r) return NULL;
4.      segment_tree *res = new segment_tree(l, r);
5.      if (l == r)
6.      {
7.          res->value = arr[l];
8.      }
9.      else
10.     {
11.         int m = (l + r) / 2;
12.         res->left = build(arr, l, m);
13.         res->right = build(arr, m + 1, r);
14.         if (res->left != NULL) res->value += res->left->value;
15.         if (res->right != NULL) res->value += res->right->value;
16.     }
17.     return res;
18. }
```

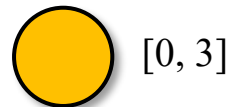
A segment tree é construída em $O(n)$, seguindo uma estratégia recursiva bottom-up.

Segment tree

0

3

```
1. segment_tree* build(const vector<int> &arr, int l, int r)
2. {
3.     if (l > r) return NULL;
4.     segment_tree *res = new segment_tree(l, r);
5.     if (l == r)
6.     {
7.         res->value = arr[l];
8.     }
9.     else
10.    {
11.        int m = (l + r) / 2;
12.        res->left = build(arr, l, m);
13.        res->right = build(arr, m + 1, r);
14.        if (res->left != NULL) res->value += res->left->value;
15.        if (res->right != NULL) res->value += res->right->value;
16.    }
17.    return res;
18. }
```



build(0, 3)

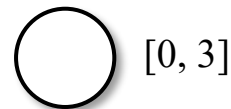
arr =	1	0	7	8
	[0]	[1]	[2]	[3]

Segment tree

0

3

```
1. segment_tree* build(const vector<int> &arr, int l, int r)
2. {
3.     if (l > r) return NULL;
4.     segment_tree *res = new segment_tree(l, r);
5.     if (l == r)
6.     {
7.         res->value = arr[l];
8.     }
9.     else
10.    {
11.        int m = (l + r) / 2;
12.        res->left = build(arr, l, m);
13.        res->right = build(arr, m + 1, r);
14.        if (res->left != NULL) res->value += res->left->value;
15.        if (res->right != NULL) res->value += res->right->value;
16.    }
17.    return res;
18. }
```



1

build(0, 3)

arr =	1	0	7	8
	[0]	[1]	[2]	[3]

Segment tree

0

3

```
1. segment_tree* build(const vector<int> &arr, int l, int r)
2. {
3.     if (l > r) return NULL;
4.     segment_tree *res = new segment_tree(l, r);
5.     if (l == r)
6.     {
7.         res->value = arr[l];
8.     }
9.     else
10.    {
11.        int m = (l + r) / 2;
12.        res->left = build(arr, l, m);
13.        res->right = build(arr, m + 1, r);
14.        if (res->left != NULL) res->value += res->left->value;
15.        if (res->right != NULL) res->value += res->right->value;
16.    }
17.    return res;
18. }
```



1

build(arr, 0, 1)

build(0, 3)

arr =	1	0	7	8
	[0]	[1]	[2]	[3]

Segment tree

0

1

```
1. segment_tree* build(const vector<int> &arr, int l, int r)
2. {
3.     if (l > r) return NULL;
4.     segment_tree *res = new segment_tree(l, r);
5.     if (l == r)
6.     {
7.         res->value = arr[l];
8.     }
9.     else
10.    {
11.        int m = (l + r) / 2;
12.        res->left = build(arr, l, m);
13.        res->right = build(arr, m + 1, r);
14.        if (res->left != NULL) res->value += res->left->value;
15.        if (res->right != NULL) res->value += res->right->value;
16.    }
17.    return res;
18. }
```



arr =

1	0	7	8
[0]	[1]	[2]	[3]

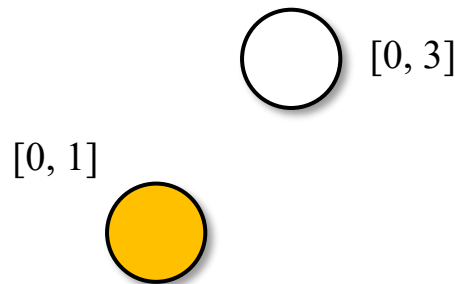
build(0, 3)
build(0, 1)

Segment tree

0

1

```
1. segment_tree* build(const vector<int> &arr, int l, int r)
2. {
3.     if (l > r) return NULL;
4.     segment_tree *res = new segment_tree(l, r);
5.     if (l == r)
6.     {
7.         res->value = arr[l];
8.     }
9.     else
10.    {
11.        int m = (l + r) / 2;
12.        res->left = build(arr, l, m);
13.        res->right = build(arr, m + 1, r);
14.        if (res->left != NULL) res->value += res->left->value;
15.        if (res->right != NULL) res->value += res->right->value;
16.    }
17.    return res;
18. }
```

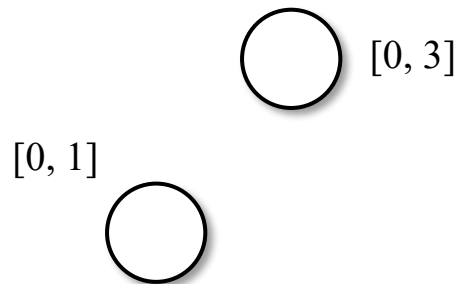


build(0, 3)
build(0, 1)

arr =	1	0	7	8
	[0]	[1]	[2]	[3]

Segment tree

```
1. segment_tree* build(const vector<int> &arr, int l, int r)
2. {
3.     if (l > r) return NULL;
4.     segment_tree *res = new segment_tree(l, r);
5.     if (l == r)
6.     {
7.         res->value = arr[l];
8.     }
9.     else
10.    {
11.        int m = (l + r) / 2;
12.        res->left = build(arr, l, m);
13.        res->right = build(arr, m + 1, r);
14.        if (res->left != NULL) res->value += res->left->value;
15.        if (res->right != NULL) res->value += res->right->value;
16.    }
17.    return res;
18. }
```



build(0, 3)
build(0, 1)

arr =

1	0	7	8
[0]	[1]	[2]	[3]

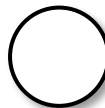
Segment tree

```
1. segment_tree* build(const vector<int> &arr, int l, int r)
2. {
3.     if (l > r) return NULL;
4.     segment_tree *res = new segment_tree(l, r);
5.     if (l == r)
6.     {
7.         res->value = arr[l];
8.     }
9.     else
10.    {
11.        int m = (l + r) / 2;
12.        res->left = build(arr, l, m);
13.        res->right = build(arr, m + 1, r);
14.        if (res->left != NULL) res->value += res->left->value;
15.        if (res->right != NULL) res->value += res->right->value;
16.    }
17.    return res;
18. }
```

build(arr, 0, 0)

[0, 1]

[0, 3]



build(0, 3)
build(0, 1)

arr =

1	0	7	8
---	---	---	---

 [0] [1] [2] [3]

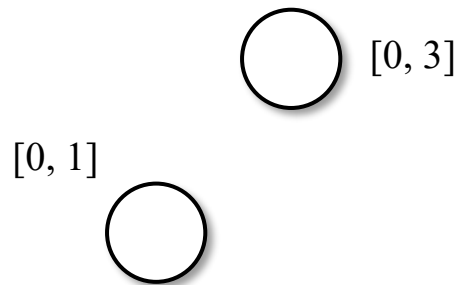
Segment tree

```
1. segment_tree* build(const vector<int> &arr, int l, int r)
2. {
3.     if (l > r) return NULL;
4.     segment_tree *res = new segment_tree(l, r);
5.     if (l == r)
6.     {
7.         res->value = arr[l];
8.     }
9.     else
10.    {
11.        int m = (l + r) / 2;
12.        res->left = build(arr, l, m);
13.        res->right = build(arr, m + 1, r);
14.        if (res->left != NULL) res->value += res->left->value;
15.        if (res->right != NULL) res->value += res->right->value;
16.    }
17.    return res;
18. }
```

arr =

1	0	7	8
---	---	---	---

[0] [1] [2] [3]



build(0, 3)
 build(0, 1)
 build(0, 0)

Segment tree

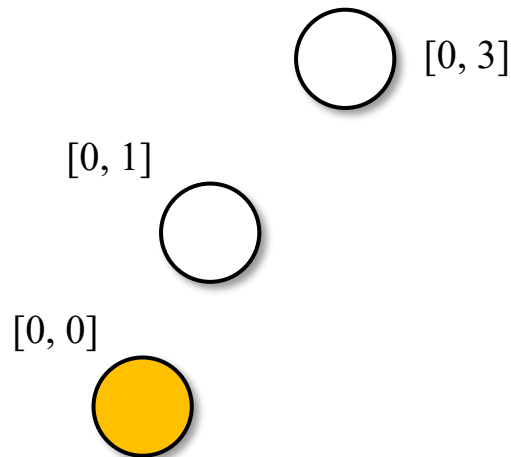
```
1. segment_tree* build(const vector<int> &arr, int l, int r)
2. {
3.     if (l > r) return NULL;
4.     segment_tree *res = new segment_tree(l, r);
5.     if (l == r)
6.     {
7.         res->value = arr[l];
8.     }
9.     else
10.    {
11.        int m = (l + r) / 2;
12.        res->left = build(arr, l, m);
13.        res->right = build(arr, m + 1, r);
14.        if (res->left != NULL) res->value += res->left->value;
15.        if (res->right != NULL) res->value += res->right->value;
16.    }
17.    return res;
18. }
```

arr =

1	0	7	8
---	---	---	---

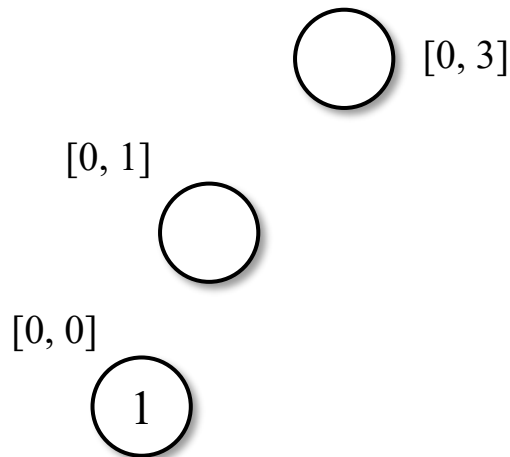
[0] [1] [2] [3]

build(0, 3)
 build(0, 1)
 build(0, 0)



Segment tree

```
1. segment_tree* build(const vector<int> &arr, int l, int r)
2. {
3.     if (l > r) return NULL;
4.     segment_tree *res = new segment_tree(l, r);
5.     if (l == r)
6.     {
7.         res->value = arr[l];
8.     }
9.     else
10.    {
11.        int m = (l + r) / 2;
12.        res->left = build(arr, l, m);
13.        res->right = build(arr, m + 1, r);
14.        if (res->left != NULL) res->value += res->left->value;
15.        if (res->right != NULL) res->value += res->right->value;
16.    }
17.    return res;
18. }
```



arr =

1	0	7	8
[0]	[1]	[2]	[3]

```
build(0, 3)
  build(0, 1)
    build(0, 0)
```

Segment tree

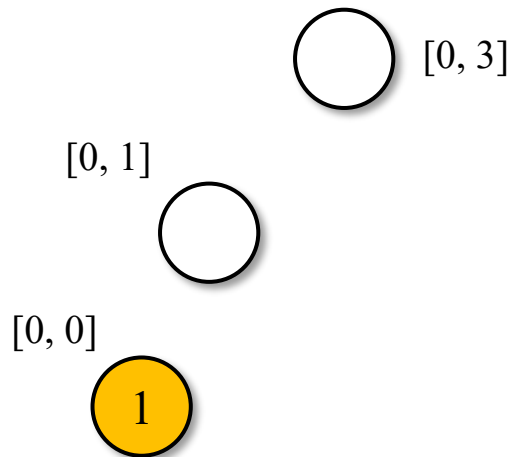
```
1. segment_tree* build(const vector<int> &arr, int l, int r)
2. {
3.     if (l > r) return NULL;
4.     segment_tree *res = new segment_tree(l, r);
5.     if (l == r)
6.     {
7.         res->value = arr[l];
8.     }
9.     else
10.    {
11.        int m = (l + r) / 2;
12.        res->left = build(arr, l, m);
13.        res->right = build(arr, m + 1, r);
14.        if (res->left != NULL) res->value += res->left->value;
15.        if (res->right != NULL) res->value += res->right->value;
16.    }
17.    return res;
18. }
```

arr =

1	0	7	8
---	---	---	---

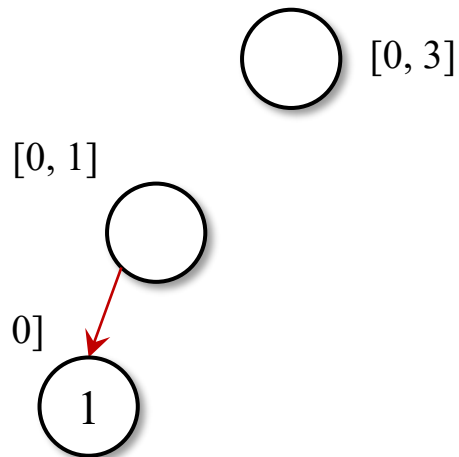
[0] [1] [2] [3]

build(0, 3)
 build(0, 1)
 build(0, 0)



Segment tree

```
1. segment_tree* build(const vector<int> &arr, int l, int r)
2. {
3.     if (l > r) return NULL;
4.     segment_tree *res = new segment_tree(l, r);
5.     if (l == r)
6.     {
7.         res->value = arr[l];
8.     }
9.     else
10.    {
11.        int m = (l + r) / 2;
12.        res->left = build(arr, l, m);
13.        res->right = build(arr, m + 1, r);
14.        if (res->left != NULL) res->value += res->left->value;
15.        if (res->right != NULL) res->value += res->right->value;
16.    }
17.    return res;
18. }
```



build(0, 3)
build(0, 1)

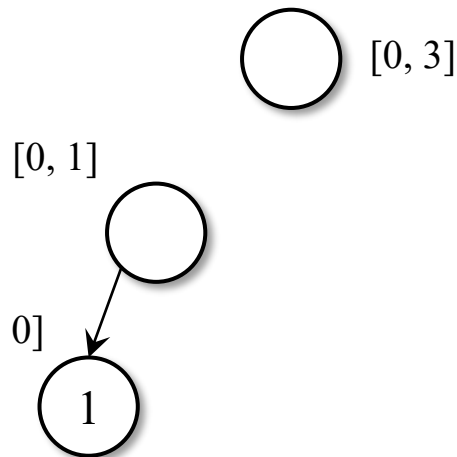
arr =

1	0	7	8
---	---	---	---

[0] [1] [2] [3]

Segment tree

```
1. segment_tree* build(const vector<int> &arr, int l, int r)
2. {
3.     if (l > r) return NULL;
4.     segment_tree *res = new segment_tree(l, r);
5.     if (l == r)
6.     {
7.         res->value = arr[l];
8.     }
9.     else
10.    {
11.        int m = (l + r) / 2;
12.        res->left = build(arr, l, m);
13.        res->right = build(arr, m + 1, r);
14.        if (res->left != NULL) res->value += res->left->value;
15.        if (res->right != NULL) res->value += res->right->value;
16.    }
17.    return res;
18. }
```



arr =

1	0	7	8
---	---	---	---

[0] [1] [2] [3]

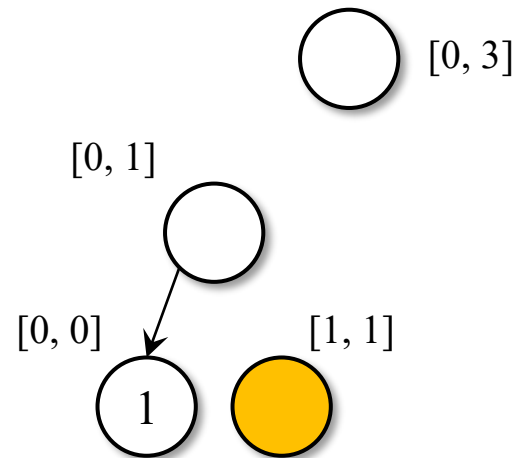
build(0, 3)
build(0, 1)

Segment tree

1

1

```
1. segment_tree* build(const vector<int> &arr, int l, int r)
2. {
3.     if (l > r) return NULL;
4.     segment_tree *res = new segment_tree(l, r);
5.     if (l == r)
6.     {
7.         res->value = arr[l];
8.     }
9.     else
10.    {
11.        int m = (l + r) / 2;
12.        res->left = build(arr, l, m);
13.        res->right = build(arr, m + 1, r);
14.        if (res->left != NULL) res->value += res->left->value;
15.        if (res->right != NULL) res->value += res->right->value;
16.    }
17.    return res;
18. }
```



arr =

1	0	7	8
---	---	---	---

[0] [1] [2] [3]

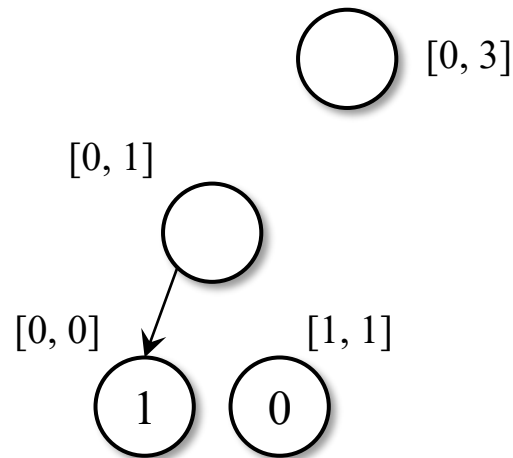
build(0, 3)
 build(0, 1)
 build(1, 1)

Segment tree

1

1

```
1. segment_tree* build(const vector<int> &arr, int l, int r)
2. {
3.     if (l > r) return NULL;
4.     segment_tree *res = new segment_tree(l, r);
5.     if (l == r)
6.     {
7.         res->value = arr[l];
8.     }
9.     else
10.    {
11.        int m = (l + r) / 2;
12.        res->left = build(arr, l, m);
13.        res->right = build(arr, m + 1, r);
14.        if (res->left != NULL) res->value += res->left->value;
15.        if (res->right != NULL) res->value += res->right->value;
16.    }
17.    return res;
18. }
```



arr =

1	0	7	8
---	---	---	---

[0] [1] [2] [3]

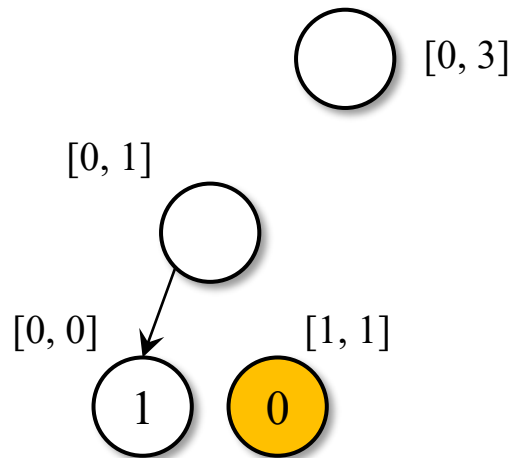
build(0, 3)
 build(0, 1)
 build(1, 1)

Segment tree

1

1

```
1. segment_tree* build(const vector<int> &arr, int l, int r)
2. {
3.     if (l > r) return NULL;
4.     segment_tree *res = new segment_tree(l, r);
5.     if (l == r)
6.     {
7.         res->value = arr[l];
8.     }
9.     else
10.    {
11.        int m = (l + r) / 2;
12.        res->left = build(arr, l, m);
13.        res->right = build(arr, m + 1, r);
14.        if (res->left != NULL) res->value += res->left->value;
15.        if (res->right != NULL) res->value += res->right->value;
16.    }
17.    return res;
18. }
```



arr =

1	0	7	8
---	---	---	---

[0] [1] [2] [3]

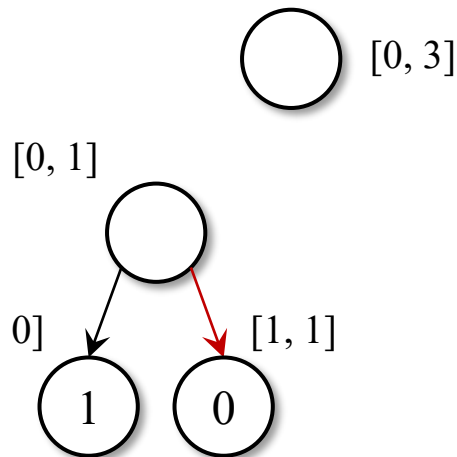
```
build(0, 3)
  build(0, 1)
    build(1, 1)
```

Segment tree

0

1

```
1. segment_tree* build(const vector<int> &arr, int l, int r)
2. {
3.     if (l > r) return NULL;
4.     segment_tree *res = new segment_tree(l, r);
5.     if (l == r)
6.     {
7.         res->value = arr[l];
8.     }
9.     else
10.    {
11.        int m = (l + r) / 2;
12.        res->left = build(arr, l, m);
13.        res->right = build(arr, m + 1, r);
14.        if (res->left != NULL) res->value += res->left->value;
15.        if (res->right != NULL) res->value += res->right->value;
16.    }
17.    return res;
18. }
```



build(0, 3)
build(0, 1)

arr =

1	0	7	8
---	---	---	---

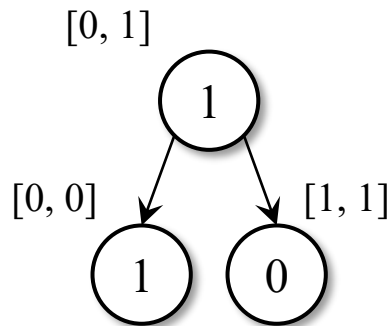
[0] [1] [2] [3]

Segment tree

0

1

```
1. segment_tree* build(const vector<int> &arr, int l, int r)
2. {
3.     if (l > r) return NULL;
4.     segment_tree *res = new segment_tree(l, r);
5.     if (l == r)
6.     {
7.         res->value = arr[l];
8.     }
9.     else
10.    {
11.        int m = (l + r) / 2;
12.        res->left = build(arr, l, m);
13.        res->right = build(arr, m + 1, r);
14.        if (res->left != NULL) res->value += res->left->value;
15.        if (res->right != NULL) res->value += res->right->value;
16.    }
17.    return res;
18. }
```



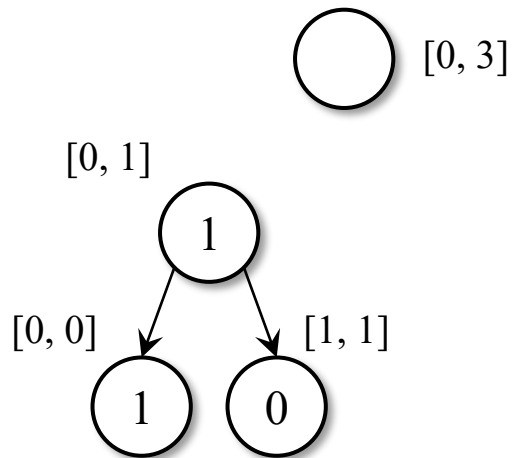
build(0, 3)
build(0, 1)

arr =

1	0	7	8
[0]	[1]	[2]	[3]

Segment tree

```
1. segment_tree* build(const vector<int> &arr, int l, int r)
2. {
3.     if (l > r) return NULL;
4.     segment_tree *res = new segment_tree(l, r);
5.     if (l == r)
6.     {
7.         res->value = arr[l];
8.     }
9.     else
10.    {
11.        int m = (l + r) / 2;
12.        res->left = build(arr, l, m);
13.        res->right = build(arr, m + 1, r);
14.        if (res->left != NULL) res->value += res->left->value;
15.        if (res->right != NULL) res->value += res->right->value;
16.    }
17.    return res;
18. }
```



build(0, 3)
build(0, 1)

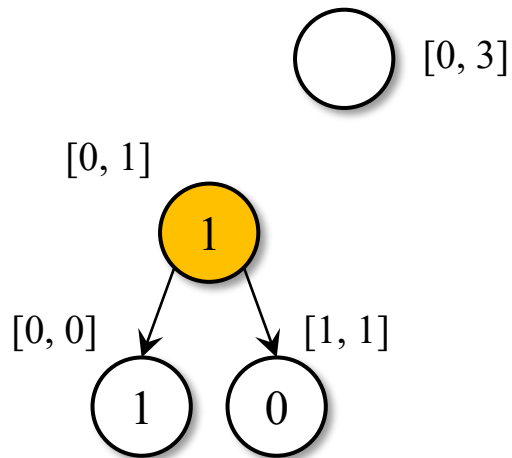
arr =

1	0	7	8
---	---	---	---

 [0] [1] [2] [3]

Segment tree

```
1. segment_tree* build(const vector<int> &arr, int l, int r)
2. {
3.     if (l > r) return NULL;
4.     segment_tree *res = new segment_tree(l, r);
5.     if (l == r)
6.     {
7.         res->value = arr[l];
8.     }
9.     else
10.    {
11.        int m = (l + r) / 2;
12.        res->left = build(arr, l, m);
13.        res->right = build(arr, m + 1, r);
14.        if (res->left != NULL) res->value += res->left->value;
15.        if (res->right != NULL) res->value += res->right->value;
16.    }
17.    return res;
18. }
```



arr =

1	0	7	8
---	---	---	---

[0] [1] [2] [3]

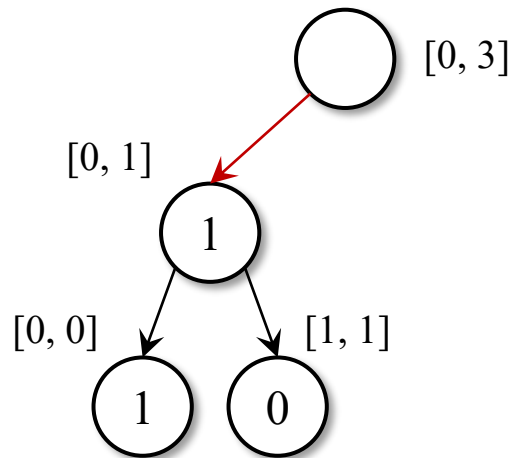
build(0, 3)
build(0, 1)

Segment tree

0

3

```
1. segment_tree* build(const vector<int> &arr, int l, int r)
2. {
3.     if (l > r) return NULL;
4.     segment_tree *res = new segment_tree(l, r);
5.     if (l == r)
6.     {
7.         res->value = arr[l];
8.     }
9.     else
10.    {
11.        int m = (l + r) / 2;
12.        res->left = build(arr, l, m);
13.        res->right = build(arr, m + 1, r);
14.        if (res->left != NULL) res->value += res->left->value;
15.        if (res->right != NULL) res->value += res->right->value;
16.    }
17.    return res;
18. }
```



build(0, 3)

arr =

1	0	7	8
---	---	---	---

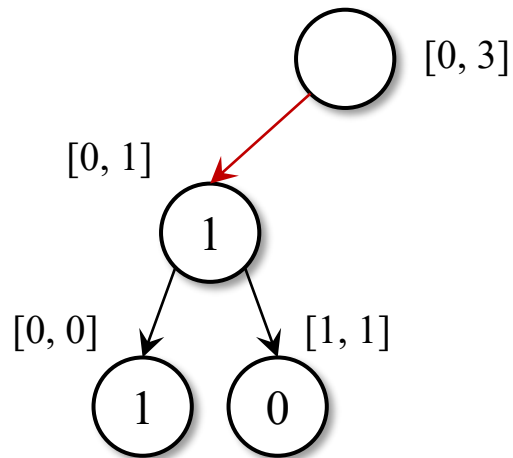
[0] [1] [2] [3]

Segment tree

0

3

```
1. segment_tree* build(const vector<int> &arr, int l, int r)
2. {
3.     if (l > r) return NULL;
4.     segment_tree *res = new segment_tree(l, r);
5.     if (l == r)
6.     {
7.         res->value = arr[l];
8.     }
9.     else
10.    {
11.        int m = (l + r) / 2;
12.        res->left = build(arr, l, m);
13.        res->right = build(arr, m + 1, r);
14.        if (res->left != NULL) res->value += res->left->value;
15.        if (res->right != NULL) res->value += res->right->value;
16.    }
17.    return res;
18. }
```



build(0, 3)

arr =

1	0	7	8
---	---	---	---

[0] [1] [2] [3]

Continua...

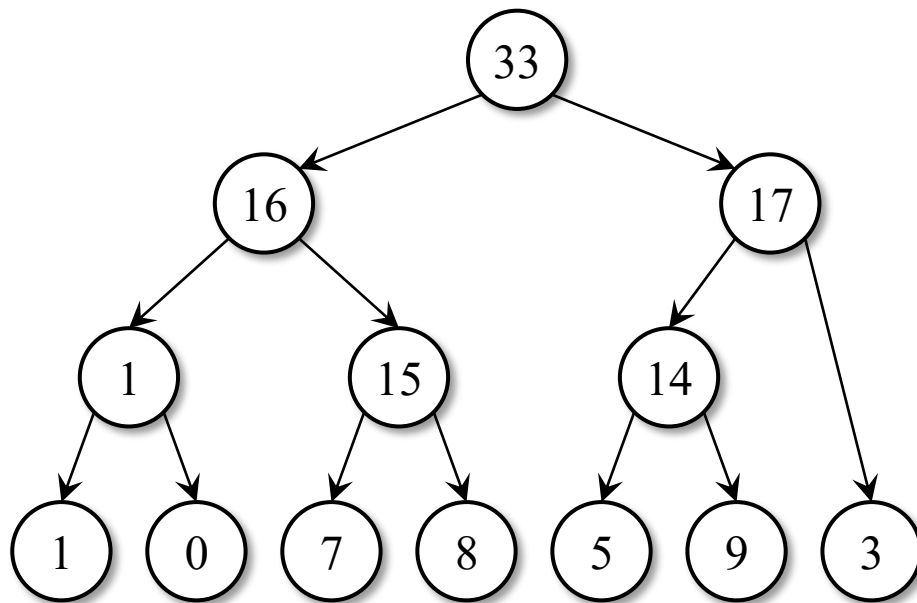
Segment tree

- ▶ Com a estrutura pronta, pode-se realizar range queries muito mais facilmente, visto que necessário considerar apenas alguns de seus nós.

Segment tree

- ▶ Com a estrutura pronta, pode-se realizar range queries muito mais facilmente, visto que necessário considerar apenas alguns de seus nós.
- ▶ Seja a consulta: $\text{sum}(0, 5) = ?$

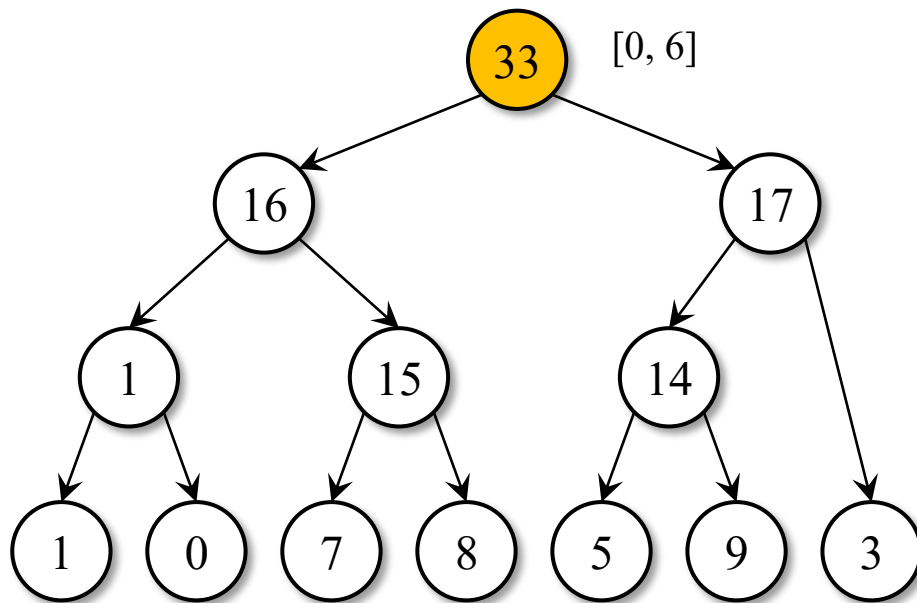
[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	8	5	9	3



Segment tree

- Com a estrutura pronta, pode-se realizar range queries muito mais facilmente, visto que necessário considerar apenas alguns de seus nós.
- Seja a consulta: $\text{sum}(0, 5) = ?$

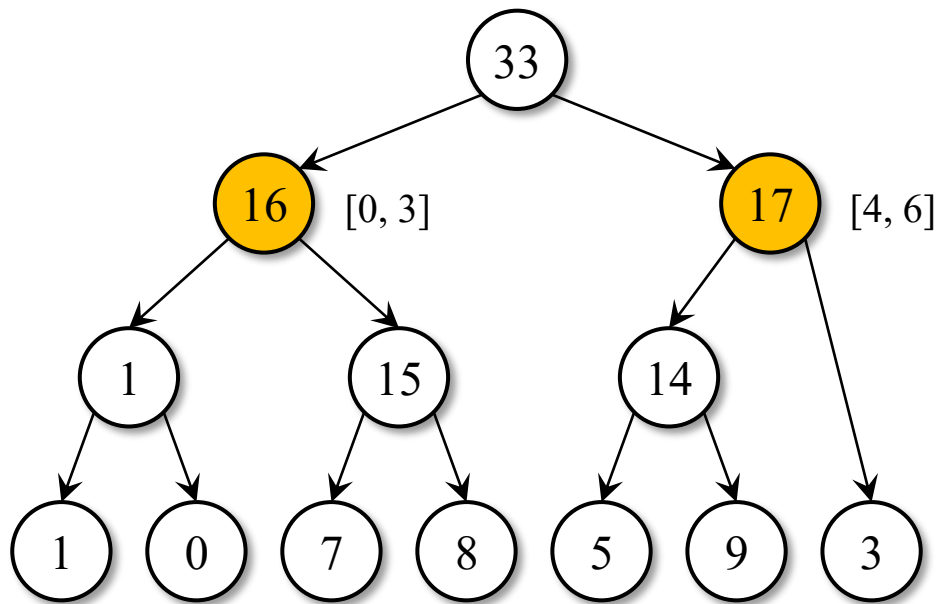
[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	8	5	9	3



Segment tree

- ▶ Com a estrutura pronta, pode-se realizar range queries muito mais facilmente, visto que necessário considerar apenas alguns de seus nós.
- ▶ Seja a consulta: $\text{sum}(0, 5) = ?$

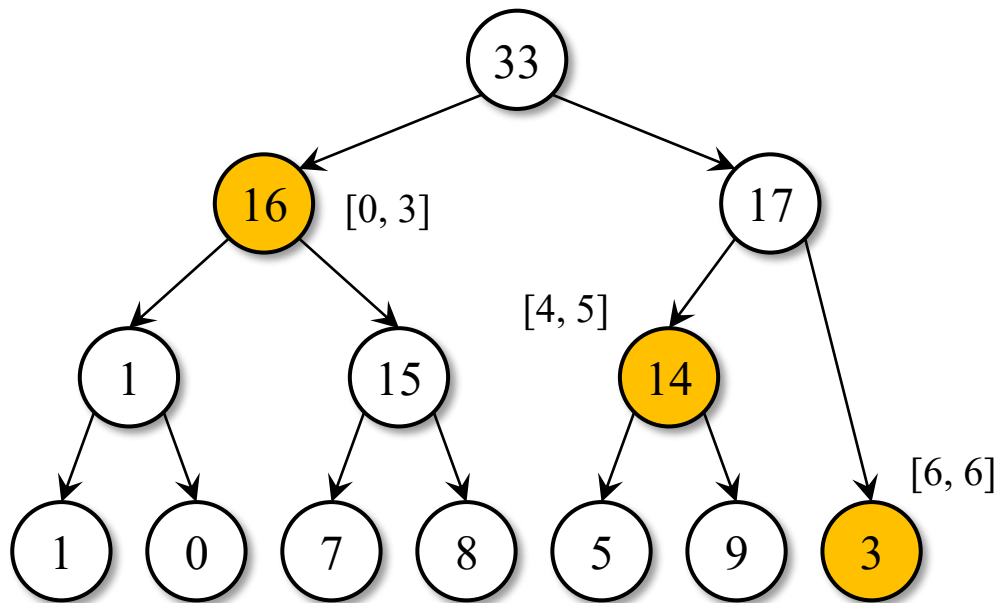
[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	8	5	9	3



Segment tree

- Com a estrutura pronta, pode-se realizar range queries muito mais facilmente, visto que necessário considerar apenas alguns de seus nós.
- Seja a consulta: $\text{sum}(0, 5) = ?$

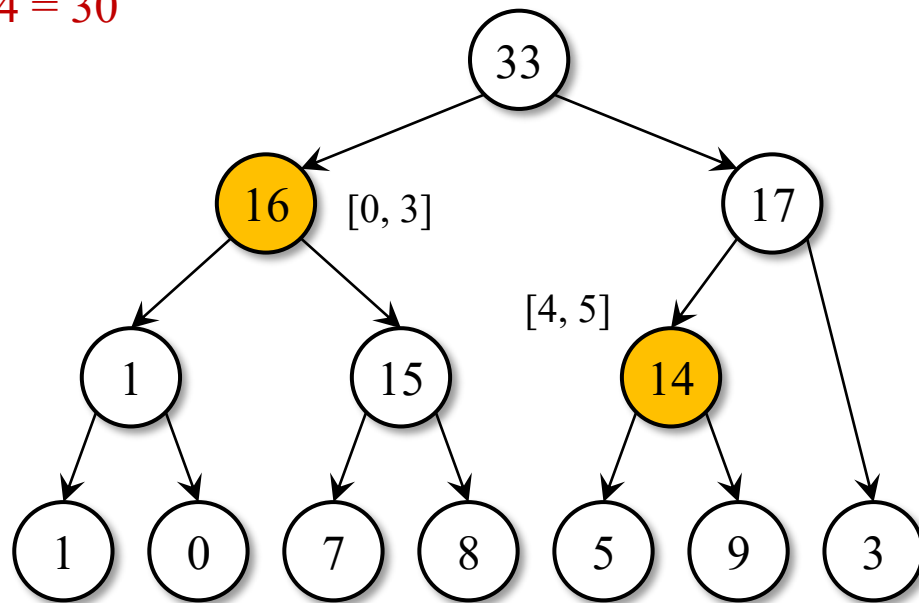
[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	8	5	9	3



Segment tree

- Com a estrutura pronta, pode-se realizar range queries muito mais facilmente, visto que necessário considerar apenas alguns de seus nós.
- Seja a consulta: $\text{sum}(0, 5) = 16 + 14 = 30$

[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	8	5	9	3



Segment tree

```
1.  int query(segment_tree *tree, int l, int r)
2.  {
3.      if (tree == NULL) return 0;
4.      if (l <= tree->from && tree->to <= r) return tree->value;
5.      if (tree->to < l) return 0;
6.      if (r < tree->from) return 0;
7.      return query(tree->left, l, r) + query(tree->right, l, r);
8.  }
```

- ▶ A função retorna um valor somente quando a consulta é realizada dentro de um segmento (limites l e r).
- ▶ Consultas fora do intervalo retornam 0. (elemento neutro da soma)
- ▶ Caso não tenhamos descido o suficiente, dividimos a query em duas.

Segment tree

```
1. int query(segment_tree *tree, int l, int r)
2. {
3.     if (tree == NULL) return 0;
4.     if (l <= tree->from && tree->to <= r) return tree->value;
5.     if (tree->to < l) return 0;
6.     if (r < tree->from) return 0;
7.     return query(tree->left, l, r) + query(tree->right, l, r);
8. }
```

- ▶ A função retorna um valor somente quando a consulta é realizada dentro de um segmento (limites l e r).
- ▶ Consultas fora do intervalo retornam 0. (elemento neutro da soma)
- ▶ Caso não tenhamos descido o suficiente, dividimos a query em duas.

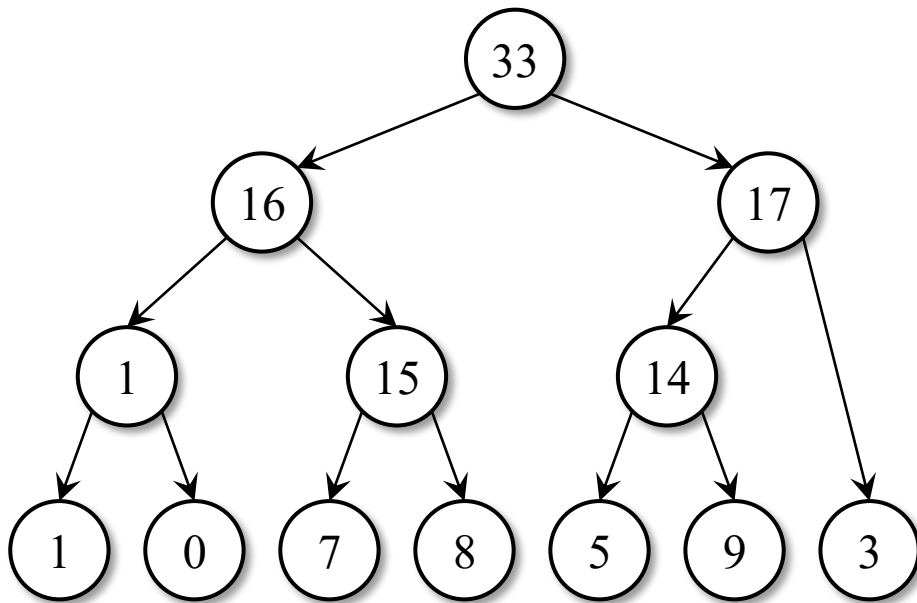
Segment tree

- ▶ Caso seja necessário **atualizar** o valor de algum elemento, a segment tree será atualizada em tempo $O(\log n)$. Este valor é bem mais rápido que $O(n)$.

Segment tree

- ▶ Caso seja necessário **atualizar** o valor de algum elemento, a segment tree será atualizada em tempo $O(\log n)$. Este valor é bem mais rápido que $O(n)$.
 - ▶ Seja a atualização: `update(3, 5)`

[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	8	5	9	3



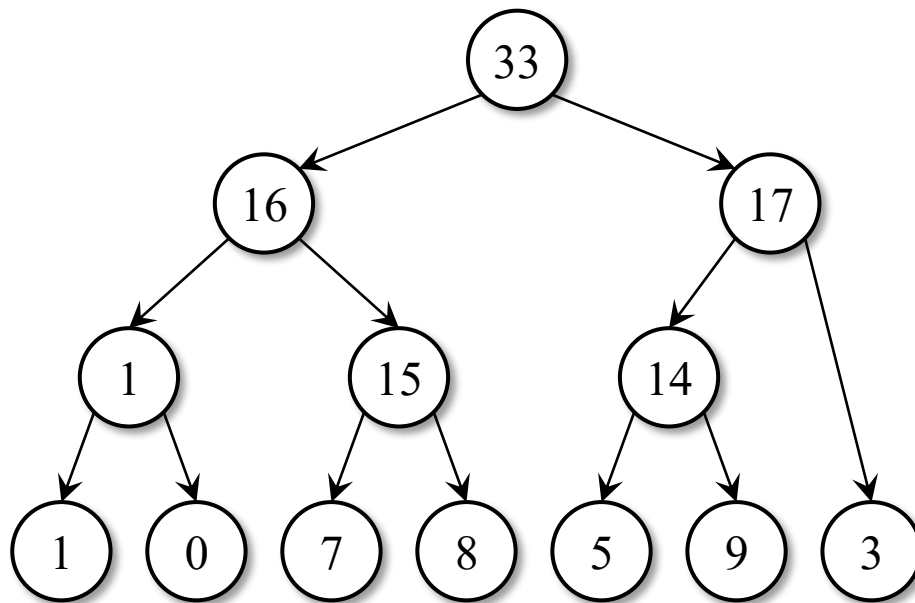
Segment tree

- ▶ Caso seja necessário **atualizar** o valor de algum elemento, a segment tree será atualizada em tempo $O(\log n)$. Este valor é bem mais rápido que $O(n)$.
 - ▶ Seja a atualização: `update(3, 5)`

[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	8	5	9	3



[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	5	5	9	3



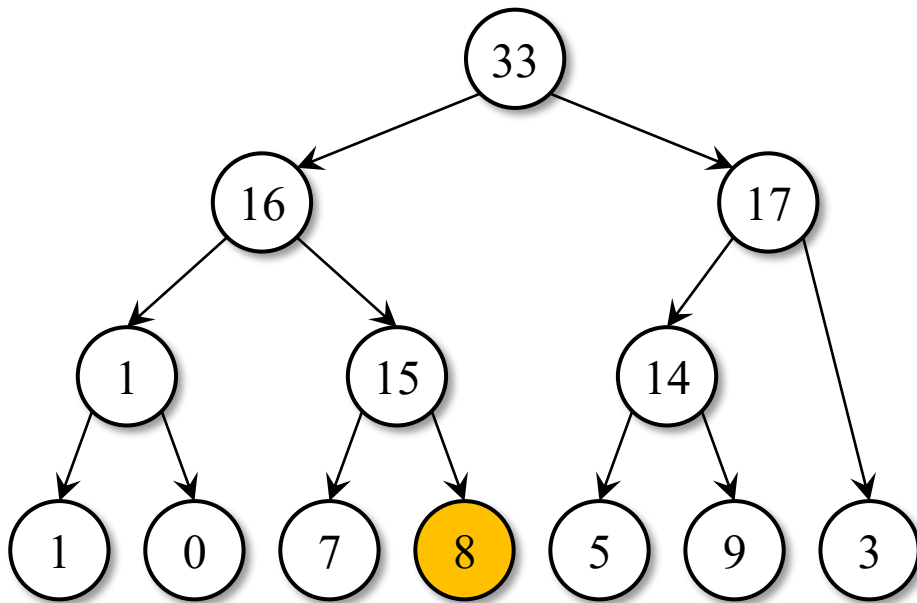
Segment tree

- ▶ Caso seja necessário **atualizar** o valor de algum elemento, a segment tree será atualizada em tempo $O(\log n)$. Este valor é bem mais rápido que $O(n)$.
 - ▶ Seja a atualização: `update(3, 5)`

[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	8	5	9	3



[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	5	5	9	3



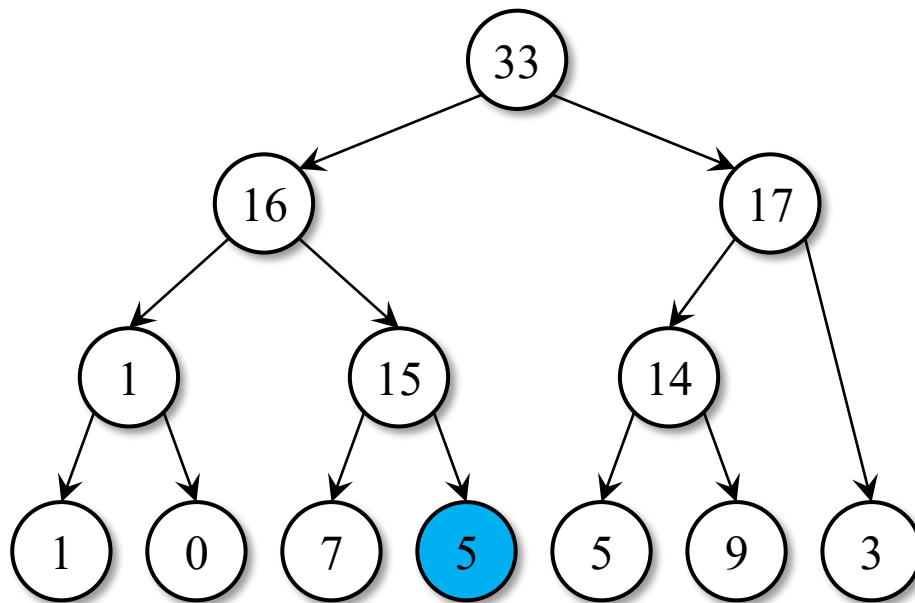
Segment tree

- ▶ Caso seja necessário **atualizar** o valor de algum elemento, a segment tree será atualizada em tempo $O(\log n)$. Este valor é bem mais rápido que $O(n)$.
 - ▶ Seja a atualização: `update(3, 5)`

[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	8	5	9	3



[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	5	5	9	3



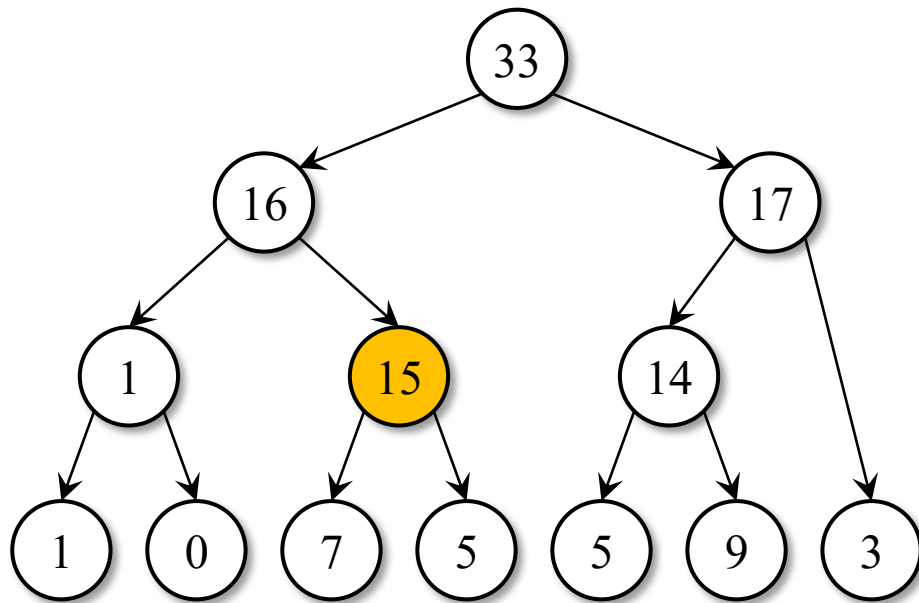
Segment tree

- ▶ Caso seja necessário **atualizar** o valor de algum elemento, a segment tree será atualizada em tempo $O(\log n)$. Este valor é bem mais rápido que $O(n)$.
 - ▶ Seja a atualização: `update(3, 5)`

[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	8	5	9	3



[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	5	5	9	3



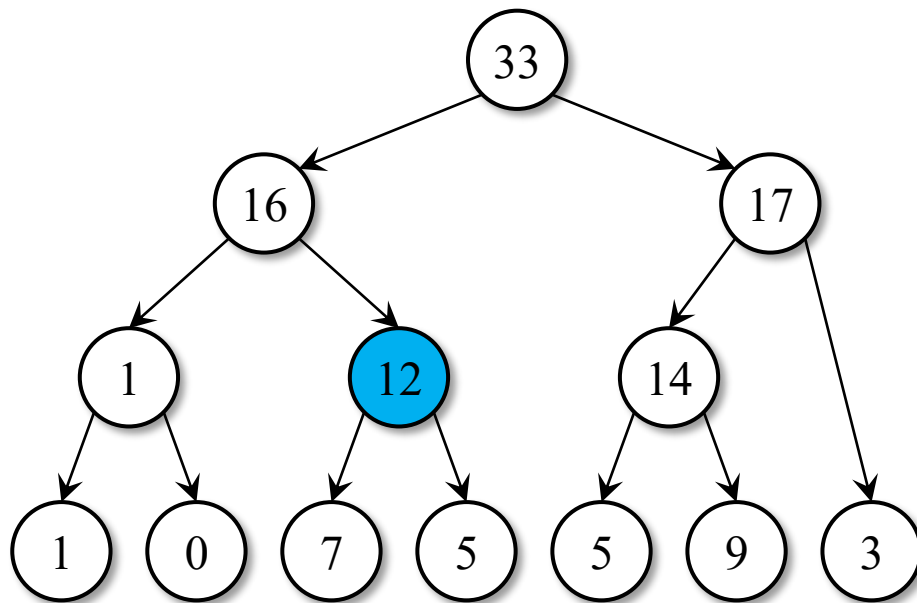
Segment tree

- ▶ Caso seja necessário **atualizar** o valor de algum elemento, a segment tree será atualizada em tempo $O(\log n)$. Este valor é bem mais rápido que $O(n)$.
 - ▶ Seja a atualização: `update(3, 5)`

[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	8	5	9	3



[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	5	5	9	3



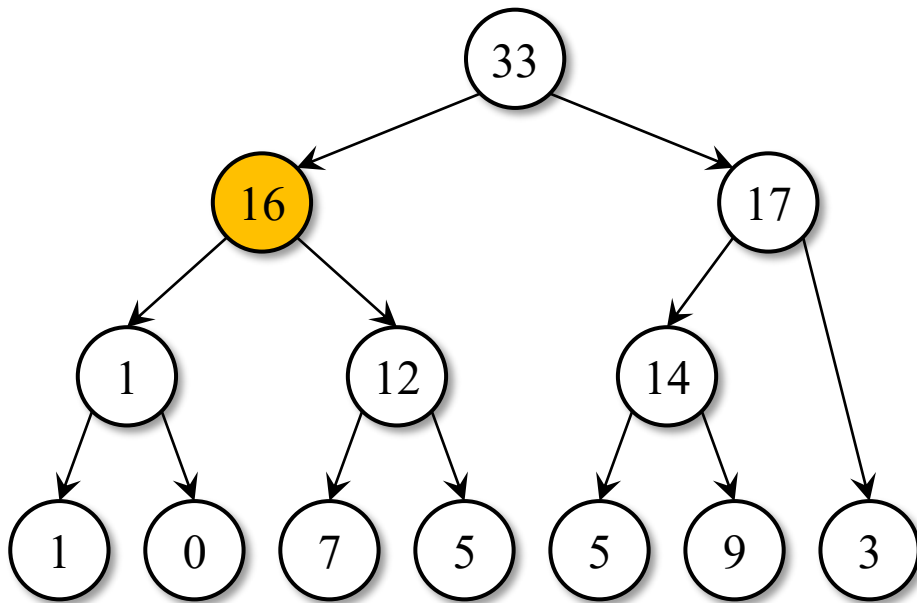
Segment tree

- ▶ Caso seja necessário **atualizar** o valor de algum elemento, a segment tree será atualizada em tempo $O(\log n)$. Este valor é bem mais rápido que $O(n)$.
 - ▶ Seja a atualização: `update(3, 5)`

[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	8	5	9	3



[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	5	5	9	3



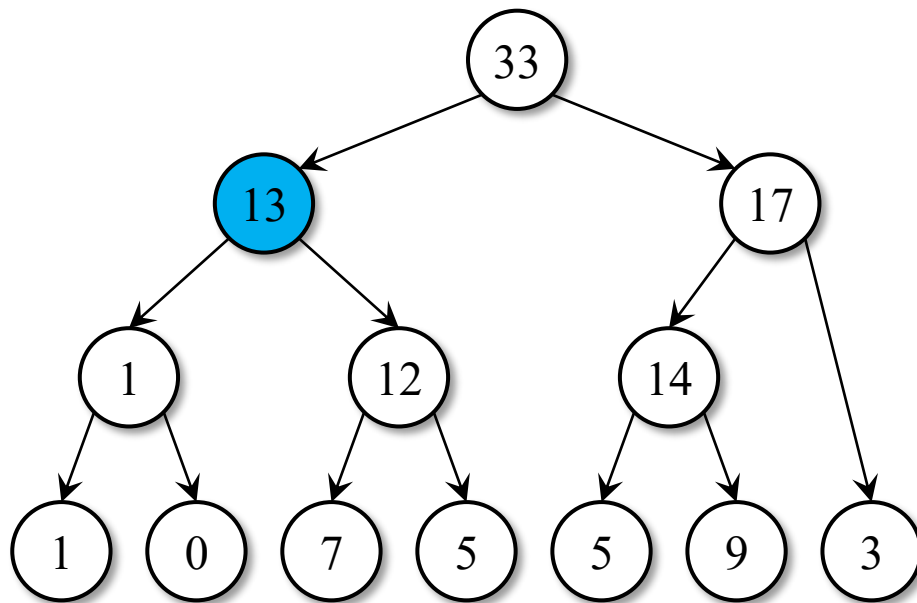
Segment tree

- ▶ Caso seja necessário **atualizar** o valor de algum elemento, a segment tree será atualizada em tempo $O(\log n)$. Este valor é bem mais rápido que $O(n)$.
 - ▶ Seja a atualização: `update(3, 5)`

[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	8	5	9	3



[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	5	5	9	3



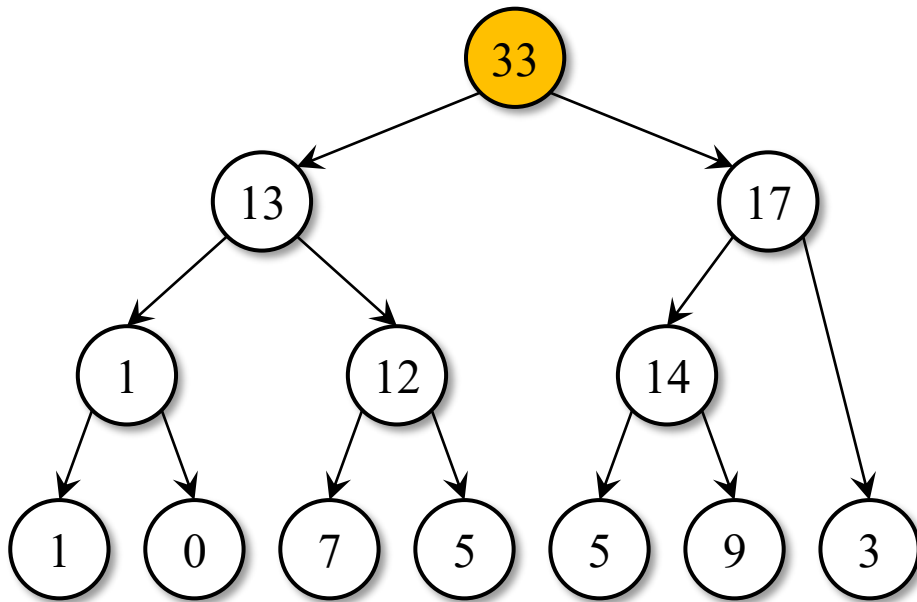
Segment tree

- ▶ Caso seja necessário **atualizar** o valor de algum elemento, a segment tree será atualizada em tempo $O(\log n)$. Este valor é bem mais rápido que $O(n)$.
 - ▶ Seja a atualização: `update(3, 5)`

[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	8	5	9	3



[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	5	5	9	3



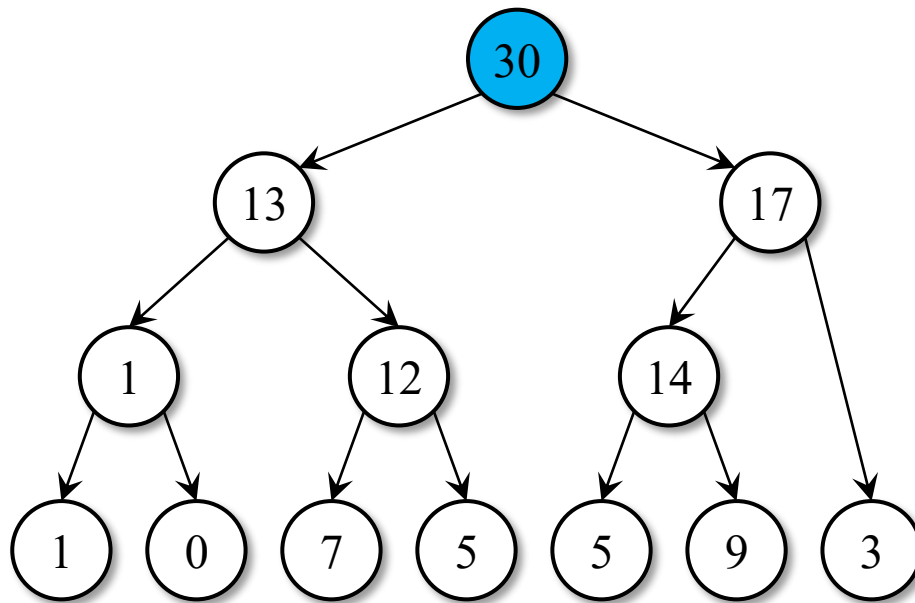
Segment tree

- ▶ Caso seja necessário **atualizar** o valor de algum elemento, a segment tree será atualizada em tempo $O(\log n)$. Este valor é bem mais rápido que $O(n)$.
 - ▶ Seja a atualização: `update(3, 5)`

[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	8	5	9	3



[0]	[1]	[2]	[3]	[4]	[5]	[6]
1	0	7	5	5	9	3



Segment tree

```
1.  int update(segment_tree *tree, int i, int val)
2.  {
3.      if (tree == NULL) return 0;
4.      if (tree->to < i) return tree->value;
5.      if (i < tree->from) return tree->value;
6.
7.      if (tree->from == tree->to && tree->from == i) tree->value = val;
8.      else tree->value = update(tree->left, i, val) +
                           update(tree->right, i, val);
9.
10.     return tree->value;
11. }
```

- ▶ Caso o nó não sofra influência de *i*, ele não é alterado.
- ▶ Caso ele seja o próprio *i*, ele é atualizado.
- ▶ Caso contrário, ele é atualizado com a soma das sub-árvores esquerda e direita atualizadas.

Segment tree

```
1.  int update(segment_tree *tree, int i, int val)
2.  {
3.      if (tree == NULL) return 0;
4.      if (tree->to < i) return tree->value;
5.      if (i < tree->from) return tree->value;
6.
7.      if (tree->from == tree->to && tree->from == i) tree->value = val;
8.      else tree->value = update(tree->left, i, val) +
                           update(tree->right, i, val);
9.
10.     return tree->value;
11. }
```

- ▶ Caso o nó não sofra influência de *i*, ele não é alterado.
- ▶ Caso ele seja o próprio *i*, ele é atualizado.
- ▶ Caso contrário, ele é atualizado com a soma das sub-árvores esquerda e direita atualizadas.

Dúvidas?



Aula 5:

Range Query

Disciplina: Maratona de Programação 1

Profs. Edmilson Marmo e Luiz Olmes

edmarmo@unifei.edu.br, olmes@unifei.edu.br



Prática

- ▶ Tarefa (para casa) disponível no Beecrowd.